TITLE: - LEGACY IO

AIM: -

To display a text by writing values to port 60h/64h

ALGORITHM: -

· Initialize the data segment.

· Display the start message.

· Read the Keyboard Controller status from port 64h and loop until the input buffer is empty (Bit 1 is 0).

· Write the command D2h (Write Output Buffer) to the Keyboard Controller command port 64h.

· Read the status from port 64h again and wait until the input buffer is empty.

· Write the target data byte ('A') to the Keyboard Controller data port 60h.

· Display the predefined text string on the screen using DOS interrupts.

· Display the success message.

· Terminate program execution.

PROGRAM:-

```
TITLE   KBCW.ASM -- Display Text by Writing to Port 60h/64h


;//****************************************************************************
;//*                                                                          *
;//*   Copyright (c) 1985-2025, American Megatrends International LLC.   *
;//*                                                                          *
;//*      All rights reserved. Subject to AMI licensing agreement.       *
;//*                                                                          *
;//****************************************************************************


;<AMI_FHDR_START>
;
; Header:
; Author: Muthuganesh S
```

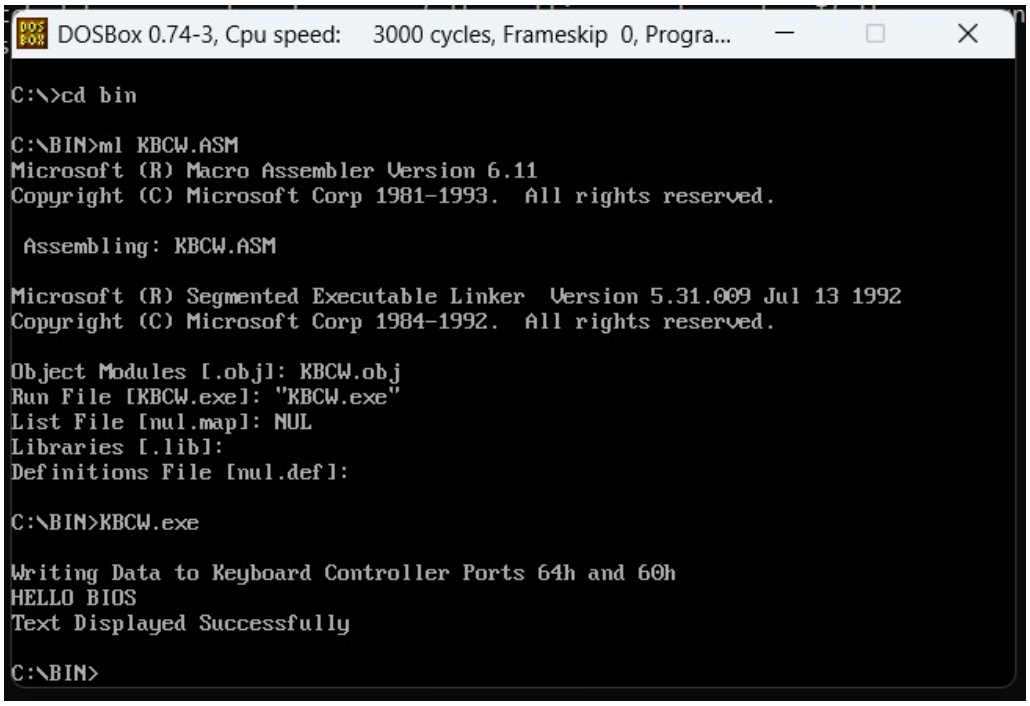
```
; Description:
; Program to send commands/data through Keyboard Controller
; ports 60h and 64h and display text on screen.
;
;<AMI_FHDR_END>

.model small
.386
.stack 100h

;--------------------------------------------------------------------------
;       DATA SEGMENT
;--------------------------------------------------------------------------
data segment

msg1 db 13,10,"Writing Data to Keyboard Controller Ports 64h and 60h",13,10,'$'
msg2 db 13,10,"Text Displayed Successfully",13,10,'$'

text db 'HELLO BIOS','$'

data ends
;--------------------------------------------------------------------------



;--------------------------------------------------------------------------
;       CODE SEGMENT
;--------------------------------------------------------------------------
code segment
assume cs:code, ds:data
```

```
start:

    mov ax,data
    mov ds,ax

    ;-------------------------------------------------
    ; Display Start Message
    ;-------------------------------------------------
    mov dx,offset msg1
    mov ah,09h
    int 21h

    ;-------------------------------------------------
    ; Wait Until Keyboard Controller Input Buffer Empty
    ; Status Port = 64h
    ; Bit1 = Input Buffer Status
    ;-------------------------------------------------
wait_input_empty:

    in al,64h
    test al,02h
    jnz wait_input_empty

    ;-------------------------------------------------
    ; Send Command to Keyboard Controller
    ; Command D2h:
    ; Write Output Buffer
    ;-------------------------------------------------
```

```
    mov al,0D2h
    out 64h,al

    ;-------------------------------------------------
    ; Wait Again
    ;-------------------------------------------------
wait_input_empty2:

    in al,64h
    test al,02h
    jnz wait_input_empty2

    ;-------------------------------------------------
    ; Send Data Byte to Port 60h
    ;-------------------------------------------------
    mov al,'A'
    out 60h,al

    ;-------------------------------------------------
    ; Display Text Normally
    ;-------------------------------------------------
    mov dx,offset text
    mov ah,09h
    int 21h

    ;-------------------------------------------------
    ; Success Message
    ;-------------------------------------------------
    mov dx,offset msg2
```

```
    mov ah,09h
    int 21h

    ;-------------------------------------------------
    ; Exit Program
    ;-------------------------------------------------
    mov ah,4Ch
    int 21h

code ends
end start
```

OUTPUT: -



AIM: -

To check and display whether the CMOS checksum is proper or not.

ALGORITHM: -

· Initialize the data segment.

· Display the start message.

· Initialize a checksum accumulator (register BX) to zero.

· Loop through CMOS memory addresses from 10h to 2Dh.

· For each address, write the address to CMOS index port 70h, read the data byte from data port 71h, and add it to the accumulator.

· Save the final accumulated value as the calculated checksum.

· Read the stored checksum high byte from CMOS address 2Eh and the low byte from address 2Fh (using ports 70h and 71h).

· Combine both bytes to form the stored 16-bit checksum.

· Display the stored CMOS checksum in hexadecimal format.

· Display the calculated CMOS checksum in hexadecimal format.

· Compare the calculated checksum with the stored checksum.

· If the checksums match, display the "PROPER" validation message.

· If the checksums do not match, display the "INVALID" validation message.

· Terminate program execution.

PROGRAM:-

```
TITLE   CHK.ASM -- Verify CMOS Checksum


;//****************************************************************************
;//*                                                                          *
;//*   Copyright (c) 1985-2025, American Megatrends International LLC.   *
;//*                                                                          *
;//*      All rights reserved. Subject to AMI licensing agreement.       *
;//*                                                                          *
;//****************************************************************************
```

```
;<AMI_FHDR_START>
;
; Header:
; Author: Muthuganesh S
; Description:
; Program to calculate CMOS checksum and verify
; whether CMOS checksum is proper or not.
;
;<AMI_FHDR_END>

.model small
.386
.stack 100h

;---------------------------------------------------------------------------
;       DATA SEGMENT
;---------------------------------------------------------------------------
data segment
msg1 db 13,10,"Checking CMOS Checksum...",13,10,'$'
msg2 db 13,10,"Stored CMOS Checksum     : $"
msg3 db 13,10,"Calculated CMOS Checksum : $"

msg_ok db 13,10,13,10,"CMOS Checksum Status : PROPER",13,10,'$'
msg_bad db 13,10,13,10,"CMOS Checksum Status : INVALID",13,10,'$'

hexbuf db '0000$'

stored_chk dw ?
```

```
calc_chk dw ?

data ends
;--------------------------------------------------------------------------



;--------------------------------------------------------------------------
;       CODE SEGMENT
;--------------------------------------------------------------------------
code segment
assume cs:code, ds:data

start:

    mov ax,data
    mov ds,ax

    ;-------------------------------------------------
    ; Display Start Message
    ;-------------------------------------------------
    mov dx,offset msg1
    mov ah,09h
    int 21h

    ;-------------------------------------------------
    ; Calculate CMOS Checksum
    ;
    ; Standard AT CMOS checksum area:
    ; 10h to 2Dh
```

```
    ;
    ; Stored checksum:
    ; 2Eh = High Byte
    ; 2Fh = Low Byte
    ;-----------------------------------------------
    xor bx,bx
    mov si,10h
checksum_loop:

    mov al,si
    out 70h,al

    in al,71h
    xor ah,ah
    add bx,ax
    inc si
    cmp si,2Eh
    jne checksum_loop

    mov calc_chk,bx

    ;-----------------------------------------------
    ; Read Stored CMOS Checksum
    ;-----------------------------------------------
    mov al,2Eh
    out 70h,al
    in al,71h

    mov ah,al
```

```
mov al,2Fh
out 70h,al
in al,71h

mov stored_chk,ax

;------------------------------------------------
; Display Stored Checksum
;------------------------------------------------
mov dx,offset msg2
mov ah,09h
int 21h

mov ax,stored_chk
call PrintHexWord

;------------------------------------------------
; Display Calculated Checksum
;------------------------------------------------
mov dx,offset msg3
mov ah,09h
int 21h

mov ax,calc_chk
call PrintHexWord

;------------------------------------------------
; Compare Checksums
```

```
    ;-------------------------------------------------
    mov ax,stored_chk
    cmp ax,calc_chk
    jne checksum_invalid

checksum_valid:

    mov dx,offset msg_ok
    mov ah,09h
    int 21h

    jmp exit_program

checksum_invalid:
    mov dx,offset msg_bad
    mov ah,09h
    int 21h
exit_program:
    ;-------------------------------------------------
    ; Exit Program
    ;-------------------------------------------------
    mov ah,4Ch
    int 21h
;--------------------------------------------------------------------------
; PrintHexWord Procedure
;--------------------------------------------------------------------------
PrintHexWord proc

    push ax
```

```
    push bx
    push cx
    push dx
    push si

    mov si,offset hexbuf
    mov cx,4

hex_loop:
    rol ax,4
    mov bl,al
    and bl,0Fh
    cmp bl,9
    jbe digit

    add bl,7

digit:

    add bl,'0'

    mov [si],bl
    inc si

    loop hex_loop

    mov dx,offset hexbuf
    mov ah,09h
    int 21h
```

```
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
PrintHexWord endp


code ends
end start
```

OUTPUT: -